Python Metaprogramming Exercises

Six exercises to consolidate the notes

Complete the exercises in order; each builds on the slides shown under its heading. Every exercise has a starter file (for example `ex1_metadata_introspection.py`) containing the code below. Replace each `# TODO`, then run the file: your output should match the expected output.

Exercise 1: Dynamic Metadata Introspection

Revised notes, slides 3 and 4

Objective: Use the attributes every class carries (__name__, __bases__, __mro__ and __dict__) to describe any class at runtime.

Instructions: Complete `inspect_class(cls)` so that it returns a dictionary with four keys: **"name"** (the class name), **"bases"** (names of the direct parents), **"mro"** (names of every class in the method resolution order) and **"attributes"** (names defined in the class itself, excluding dunder names such as __module__).

Starter file: `ex1_metadata_introspection.py`

```
def inspect_class(cls):
    """Return a summary dictionary describing any class."""
    # TODO: build and return the dictionary described above
    pass


if __name__ == "__main__":
    class Parent:
        category = "base"

    class Child(Parent):
        value = 100

        def run(self):
            pass

    meta = inspect_class(Child)
    print("Class name:", meta["name"])
    print("Bases:", meta["bases"])
    print("MRO:", meta["mro"])
    print("Own attributes:", sorted(meta["attributes"]))
```

Expected output:

```
Class name: Child
Bases: ['Parent']
MRO: ['Child', 'Parent', 'object']
Own attributes: ['run', 'value']
```

Exercise 2: Dynamic Configuration Wrapper

Revised notes, slides 5 and 6

Objective: Use __getattr__ to read a dictionary with dot notation, including nested sections.

Instructions: Complete `__getattr__` in `ReadOnlyConfig` so that `config.debug` returns the value of the "debug" key, nested dictionaries can be read with dots (`config.database.host`), and a missing key raises `AttributeError`. The class is read-only: `__setattr__` is already written to block every assignment.

Starter file: `ex2_config_wrapper.py`

```
class ReadOnlyConfig:
    def __init__(self, data):
        # Our own __setattr__ blocks writes, so store _data directly
        object.__setattr__(self, "_data", dict(data))

    def __getattr__(self, name):
        # TODO: look up name in self._data as described above
        pass

    def __setattr__(self, name, value):
        raise AttributeError("ReadOnlyConfig is read-only.")


if __name__ == "__main__":
    config = ReadOnlyConfig({
        "debug": True,
        "database": {"host": "localhost", "port": 5432},
    })
    print("Debug:", config.debug)
    print("Database host:", config.database.host)
    print("Database port:", config.database.port)

    try:
        config.timeout
    except AttributeError as err:
        print("Missing key:", err)

    try:
        config.debug = False
    except AttributeError as err:
        print("Write blocked:", err)
```

Expected output:

```
Debug: True
Database host: localhost
Database port: 5432
Missing key: No setting called 'timeout'
Write blocked: ReadOnlyConfig is read-only.
```

Exercise 3: Strict Attribute Validation

Revised notes, slides 7 and 8

Objective: Use __setattr__ to check and tidy every value before it is stored.

Instructions: Complete `__setattr__` in `StrictEmployee`. **name** must be a string (otherwise raise `TypeError`) and must not be empty once spaces are stripped (otherwise raise `ValueError`); store the stripped name. **salary** must be an int or a float (`TypeError`) and must not be negative (`ValueError`); store it as a float. The assignments in `__init__` pass through `__setattr__` too.

Starter file: `ex3_strict_validation.py`

```
class StrictEmployee:
    def __init__(self, name, salary):
        self.name = name          # both assignments go through __setattr__
        self.salary = salary

    def __setattr__(self, name, value):
        # TODO: validate and normalise, then store with object.__setattr__
        pass


if __name__ == "__main__":
    emp = StrictEmployee("  Alice ", 75000)
    print(f"Employee registered: {emp.name}, salary: {emp.salary}")

    for bad_name, bad_salary in [("   ", 50000), ("Bob", -10), (42, 50000)]:
        try:
            StrictEmployee(bad_name, bad_salary)
        except (TypeError, ValueError) as err:
            print(f"{type(err).__name__}: {err}")
```

Expected output:

```
Employee registered: Alice, salary: 75000.0
ValueError: Employee name cannot be empty
ValueError: Salary cannot be negative
TypeError: Employee name must be a string
```

Exercise 4: Generating a Class with type()

Revised notes, slides 9 and 10

Objective: Use the three-argument form of type() to build a new class while the program runs.

Instructions: Complete `class_factory(class_name, fields)` so that it returns a new class. Its `__init__` takes one value per field, in order, and stores each one (raise `TypeError` if the number of values is wrong). Its `__repr__` shows the values, for example `Point3D(x=10, y=20, z=30)`.

Starter file: `ex4_class_factory.py`

```
def class_factory(class_name, fields):
    """Build a new class whose __init__ sets the given fields in order."""
    # TODO: define __init__ and __repr__, then build the class with type()
    pass


if __name__ == "__main__":
    Point3D = class_factory("Point3D", ["x", "y", "z"])
    p = Point3D(10, 20, 30)
    print("Class name:", Point3D.__name__)
    print("Coordinates:", p.x, p.y, p.z)
    print("Object:", p)

    try:
        Point3D(1, 2)
    except TypeError as err:
        print("Error:", err)
```

Expected output:

```
Class name: Point3D
Coordinates: 10 20 30
Object: Point3D(x=10, y=20, z=30)
Error: Point3D expects 3 values, got 2
```

Exercise 5: Dynamic API Method Generator

Revised notes, slides 11 and 12

Objective: Generate one method per endpoint from a list of names, instead of writing each method by hand.

Instructions: Complete `make_method(endpoint)` and `generate_api_client(endpoints)`. The result must be a subclass of `ApiClient` with one method per endpoint, named `get_<endpoint>` (for example `get_users`), that returns `self.mock_get(endpoint)`.

Starter file: `ex5_api_method_generator.py`

```
class ApiClient:
    def __init__(self, base_url):
        self.base_url = base_url

    def mock_get(self, endpoint):
        return f"GET request succeeded for {self.base_url}/{endpoint}"


def make_method(endpoint):
    """Return a method that calls mock_get for one fixed endpoint."""
    # TODO: define and return the method
    pass


def generate_api_client(endpoints):
    """Return a subclass of ApiClient with one get_<endpoint> method each."""
    # TODO: build a namespace dict of methods, then use
    #       type("DynamicApiClient", (ApiClient,), namespace)
    pass


if __name__ == "__main__":
    Client = generate_api_client(["users", "orders"])
    client = Client("https://api.system.internal")
    print("Users API:", client.get_users())
    print("Orders API:", client.get_orders())
    print("Generated methods:",
          sorted(n for n in vars(Client) if n.startswith("get_")))
```

Expected output:

```
Users API: GET request succeeded for https://api.system.internal/users
Orders API: GET request succeeded for https://api.system.internal/orders
Generated methods: ['get_orders', 'get_users']
```

Exercise 6: Metaclass Interface Verifier

Revised notes, slides 13 to 16

Objective: Use a metaclass to reject badly formed classes at the moment they are defined.

Instructions: Complete `InterfaceVerifier.__new__` so that every subclass of `BaseComponent` must have a non-empty docstring and a `validate()` method; otherwise raise `TypeError` with the messages shown in the expected output. `BaseComponent` has no parents, so it is skipped; do not exempt classes by name. **Extension:** apply the same rules in `SimpleBase.__init_subclass__`, without a metaclass.

Starter file: `ex6_metaclass_verifier.py`

```
class InterfaceVerifier(type):
    def __new__(mcls, name, bases, namespace):
        # TODO: if bases is not empty, check the docstring and validate()
        return super().__new__(mcls, name, bases, namespace)


class BaseComponent(metaclass=InterfaceVerifier):
    pass


# Extension: the same rules with __init_subclass__, no metaclass needed
class SimpleBase:
    def __init_subclass__(cls, **kwargs):
        super().__init_subclass__(**kwargs)
        # TODO (extension): apply the same two checks to cls


if __name__ == "__main__":
    class DatabaseComponent(BaseComponent):
        """Stores records in the database."""
        def validate(self):
            return True

    DatabaseComponent()
    print("DatabaseComponent created successfully.")

    try:
        class BrokenDoc(BaseComponent):
            def validate(self):
                return True
    except TypeError as err:
        print("Docstring check failed correctly:", err)

    try:
        class BrokenMethod(BaseComponent):
            """Has a docstring but no validate()."""
    except TypeError as err:
        print("Method check failed correctly:", err)

    try:
        class Base(BaseComponent):    # the name "Base" gives no exemption
            pass
    except TypeError as err:
        print("No loophole:", err)

    try:
        class AlsoBroken(SimpleBase):
            """Docstring present, validate() missing."""
    except TypeError as err:
        print("Extension check:", err)
```

Expected output:

```
DatabaseComponent created successfully.
Docstring check failed correctly: Class 'BrokenDoc' must define a non-empty docstring.
Method check failed correctly: Class 'BrokenMethod' must implement a 'validate(self)' method.
No loophole: Class 'Base' must define a non-empty docstring.
Extension check: Class 'AlsoBroken' must implement a 'validate(self)' method.
```

The last two lines appear only once the extension and the "Base" check are working.